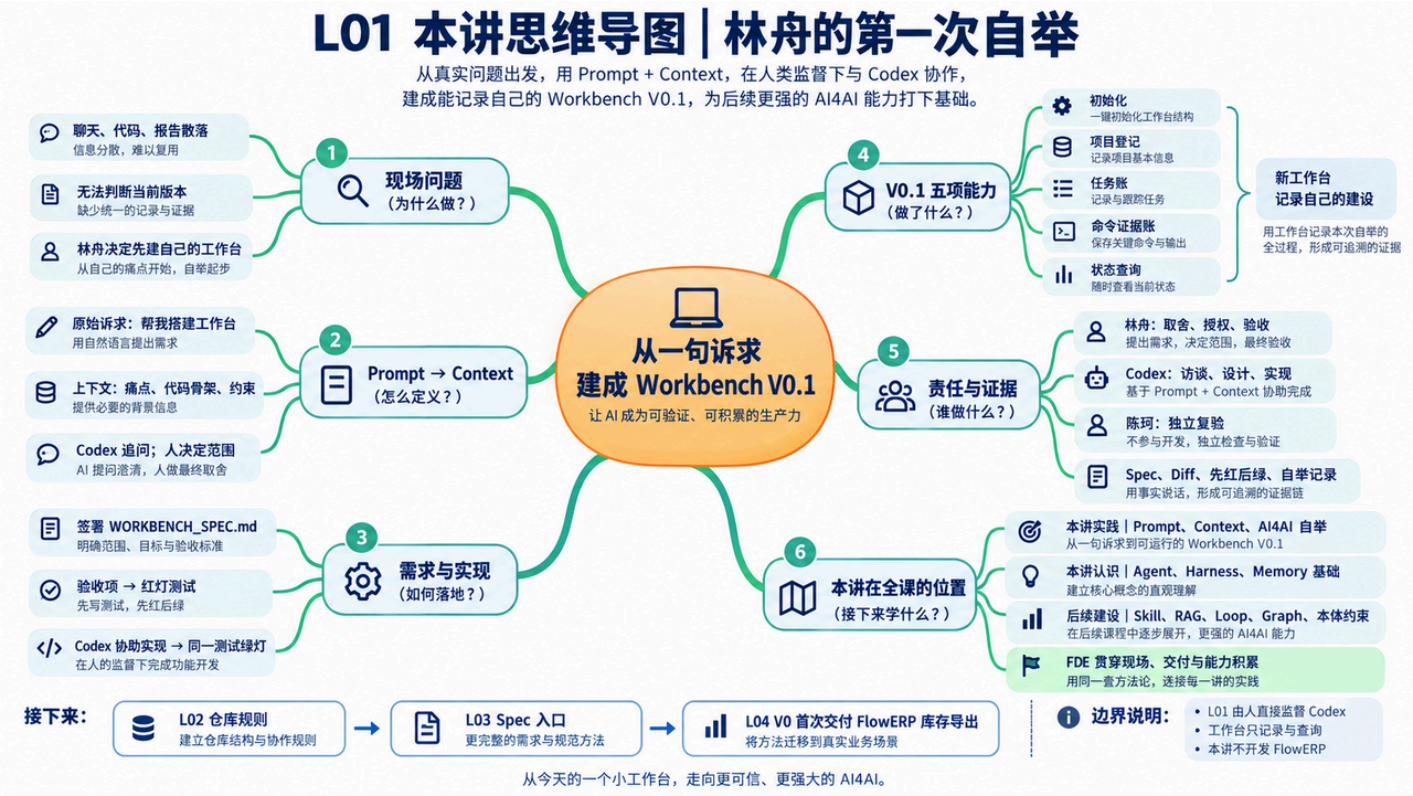
L01-辅导资料｜以终为始：一次可验证的 AI 交付怎样完成?

> **本资料由 AI 辅助整理，为课程赠送资料，用来辅助你理解课程内容，可作为查询手册使用，非必读资料。**

导读一：本讲要完成什么

本讲从一句“帮我搭建个人 AI 研发工作台”出发，由你确认需求，Codex 协助写测试和实现，最后用新工作台查回自己的建设过程。完整主线是：需求取舍 → 签署 Spec → 设计记录关系 → 先红后绿 → 自举复核。交付物包括本人签署的需求、五个可运行操作、改动与测试记录，以及人工验收结论。

导读二：本讲思维导图



先读中心目标，再沿现场问题、需求、实现和自举理解本讲。图中的全课能力用于定位后续学习；今天由你直接监督 Codex 建设 Workbench V0.1，工作台负责记录与查询。

导读三：林舟的项目手记

林舟准备持续改进 FlowERP，运营周宁想先拿到库存导出。他却发现自己连上周的小工具能否交付都说不清：聊天写着“已完成”，终端有通过记录，代码后来又改过。

陈珂准备帮他复验，问：“我该检查哪一版？”林舟决定先搭一个能登记项目、任务和运行记录的小工作台。他向 Codex 提出诉求，先谈清为什么需要、先做多少、怎样验收。

林舟、周宁和陈珂是用于串联案例的虚构人物。图中参考程序与文中预期行为均不能替代你自己的运行结果。

一、从接手困难确定第一版范围

要决定工作台先做什么，先看林舟为什么无法判断旧任务能否交付。问题不在于没有材料，而在于要求、代码与检查结果没有对应起来。

回到林舟那次开发，上午他提出：“帮我做一个任务记录工具。”讨论后补充了两个要求：关闭程序以后仍能查到任务；同一个编号不能覆盖已有任务。Codex 给出实现，林舟运行创建命令，终端显示成功。

下午，林舟发现关闭程序后任务消失了。Codex 修改保存方式后，他再次检查，这次通过。随后他又调整了错误提示，却没有重新测试，就结束了当天的工作。

这些材料都可能是真的，但林舟仍然无法立即回答。通过的是最后修改后的版本吗？当时测试是否真的关闭了程序？重复编号会不会破坏旧任务？这些问题的答案散落在不同位置，还需要他重新拼接。

持续开发要保存文件，也要保存要求、操作、结果与版本之间的关系。否则“最近一次测试通过”很容易被误当成“当前版本已经满足本次要求”。

聊天适合讨论，Git 保存代码版本与改动，文件可以存放需求和输出。这些工具各自有用，困难在于连接：哪段讨论成为正式决定，哪份输出检查哪次修改，哪些失败还没解决？工作台要把这些对应关系保存下来，使人能从一项任务继续工作。

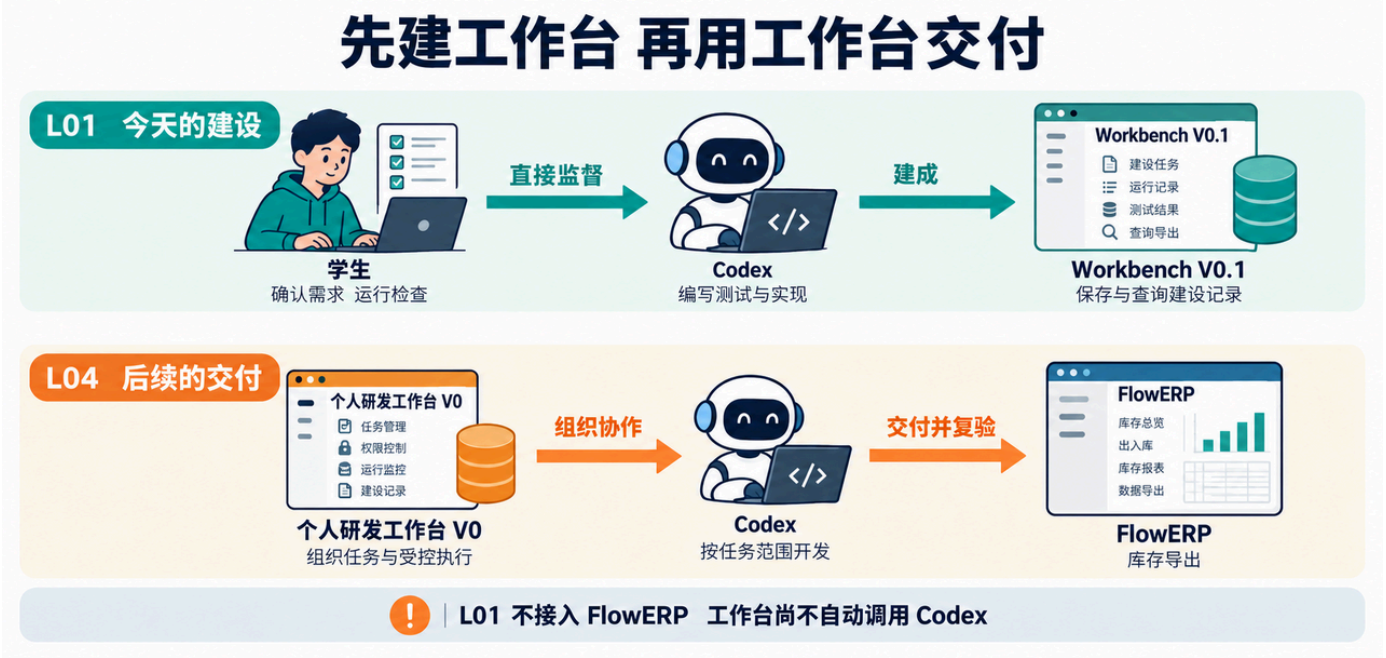
建设应用也有成本：要维护数据、处理例外、支持中断恢复。如果文件和命名约定已经足以组织少量任务，暂时沿用它们也合理。本课程选择开发工作台，是为了在持续开发 FlowERP 时反复检验这套组织方法。

验证练习：请同伴只凭现有材料找出原要求、最近的有效检查和下一项待办。记下哪些信息仍需口头补充。以后用工作台接手难度相近的任务时，再观察困难是否减少；同时考虑熟悉程度和任务差异，不能把所有变化都归因于工具。

1.1 本讲先交付记录与查询能力

课程先用 Codex 搭建个人研发工作台，再通过工作台持续开发 FlowERP。L01～L03 由你直接监督 Codex 建设；L04 补齐 Workbench V0，才首次通过它交付 FlowERP 库存导出。本讲 ERP 业务状态不变。

L01 的五个命令尚不自动调用 Codex，由你先组织开发与检查。第一版先选择任务与记录，是因为后续无论怎样执行，都需要回答“按什么要求做，结果属于哪次工作”。先建立这层关系，再接入执行与检查，自动化才有明确的输入和可以回查的结果。



课程建设边界：L01 由人组织开发；L04 首次通过工作台交付 FlowERP

二、从需求访谈形成可验收的 Spec

林舟没有先选数据库。他先把“下次能接着做”拆成可以检查的要求：重启后能查回任务，同一编号不能覆盖旧记录，每条运行结果能够找到所属任务。Codex 可以建议实现方案，先做哪些、怎样接受仍由他决定。

2.1 先分清使用目标和实现办法



六阶段交付链路：验收依据与使用效果分别检查

从业务需求到产品要求，再到技术方案、开发、验收与效果回收，各环节回答不同的问题。读图时先抓住本次任务：为什么做、做成什么、怎样实现、怎样验证、投入使用后有没有帮助。

为什么要从业务问题开始？因为“增加一个查询按钮”已经选定了解法，却还没说明谁遇到什么困难。同样，“测试全部通过”说明实现满足了已检查的约定，还需要观察使用者是否因此更容易完成工作。

产品需求文档（PRD）说明谁需要什么结果及其价值。**技术方案**说明怎样实现，包括数据、接口、依赖、影响范围和取舍。执行任务还需要一份足够具体的 **Spec（需求说明）**，将本次目标、范围和验收条件固定下来。小任务可以把这些内容放在一份文档里，但仍要分别想清楚。

例如，“接手更方便”是产品目标，“重启后按编号查回原需求”是一项可验收行为，“把需求全文写入任务表”是一种实现办法。这三个层次有关联，不能互相替代。采用了数据库不代表实现正确，实现正确也不一定解决了全部接手问题。

测试之后还要经过相应的验收与发布，才能进入真实使用。这里的“线上效果回收”指收集投入使用后的表现；对个人本地工作台，就是在自己的日常开发中使用并回收结果，不要求先部署到公网。预先说明观察对象、时间范围和比较口径，才能解释结果。任务更简单、使用者更熟练，也可能让接手变快，不能把全部变化都归因于工具。

2.2 由谁提出建议 由谁作出决定

工作台还没有建好时，由你直接组织开发。先说清原始问题和个人设想，让 Codex 追问隐藏条件；你比较建议、确定范围，再让 Codex 整理 Spec。Spec 是写清本次目标、范围和验收条件的需求说明。

时点	Codex 的协助	你要完成的判断
需求访谈	追问场景，提出可选方案	选择先解决什么，说明理由
形成 Spec	整理输入、输出、失败行为与验收项	核对并签署具体版本
准备测试	将验收项转成可运行检查	确认检查经过真实行为且没有漏掉关键条件
实现与修订	在授权范围内编写和修复代码	核对实际改动，亲自运行正式复验
最终复核	整理记录与待办	结合同伴反馈，决定接受、退回或证据不足

2.3 把范围和失败行为写清楚

本课程把写清目标、范围和验收条件的需求说明称为 **Spec**。下面是其中一个验收项的节选，完整文档还要覆盖五个操作：

代码块

```
项目：个人研发工作台
目标：创建任务后可以按编号查回要求
约定：任务必须属于已登记项目；关闭程序后仍能读取
重复编号：本版拒绝重复创建，不覆盖原任务
验收：创建任务 A，关闭程序后查询，原要求仍在
反例：再次用 A 提交不同内容，应拒绝，原要求与记录不变
```

需求范围确定以后，还要落实到文件。**写集，就是本次允许修改的具体文件集合**；本课程用这个词描述执行范围。例如，实现工作台命令与对应测试，不包含修改 FlowERP 业务代码。发现额外依赖时，先说明原因，再调整范围。

验收项要写清前提、操作、响应和操作后状态。例如，重复编号请求应被拒绝，原任务及记录还应保持不变；只写“返回错误”，就会漏掉“报错前已经覆盖”的缺陷。

三、用数据关系和五个操作实现需求

需求决定要保存什么。只留摘要会丢失检查依据，只留孤立日志又需要重新拼接。下面从一项任务出发，确定对象、接口与失败时的数据规则。

3.1 项目 任务与运行记录怎样关联

“个人研发工作台”是一个长期维护的项目；“开发 V0.1”是本次任务；实现前失败与实现后通过，是同一任务的两次运行。以后增加自动执行能力时，应建立另一项任务，保留 V0.1 的建设历史。



对象关系：项目与任务以及任务与运行记录均为一对多

图中的需求快照属于任务创建时的约定，运行记录按次追加。不能用项目上的“最近一次通过”覆盖各项任务自己的历史。

项目与任务采用稳定编号，也就是 **ID**。名称可以调整，编号仍用于关联原对象。任务还应保存当时需求内容的副本，称为**需求快照**。

例如，创建任务 A 时，Spec 约定“拒绝重复编号”，你据此实现并验收。一周后，新任务提出“相同内容重复提交时返回原结果”，同一个 Spec 文件随之更新。若 A 只保存文件路径，再打开它就会看到新规则，旧实现会被误判为没有遵守当时的要求。

需求快照保存 A 创建时的原文，使旧检查仍能对应旧约定。后续需求可以变化，但不能把变化后的文字当成过去已经采用的依据。快照帮助追溯版本，并不自行提供防篡改或身份认证。

3.2 五个命令分别负责什么

这版程序称为 **V0.1**，提供五个操作：

操作	形成或读取的信息	实践命令
初始化工作台	所有者、工作台身份、数据目录	workbench-init
登记项目	项目名称与代码位置	workbench-project-add
创建任务	项目归属、本次要求及其快照	workbench-task-create
追加运行记录	实际命令、输出与结果	workbench-evidence-add
查看状态	读取已有材料，指出缺失项	workbench-status

V0.1 的交付形式是 Python 命令行程序（CLI，Command-Line Interface）。 五个名称是 workbench.cli 的子命令：你在终端发出请求，程序处理并输出结果。验收会实际运行命令，仅提供 Python 函数不够；本讲不要求网页或 HTTP 接口。

五个名称是 workbench.cli 的子命令。完整参数、固定路径、预期输出与排错见实践操作手册。实现后，状态查询的入口如下：

代码块

```
python -X utf8 -m workbench.cli workbench-status
```

前半部分启动工作台的命令行入口，最后的 workbench-status 选择查询操作；其余参数和预期输出见实践手册。**持久化**指程序结束后数据仍保留，下次启动可以读取。本课使用本地 SQLite 保存记录，Codex 可以协助实现具体存取代码。

3.3 持久保存与失败后状态怎样保证

设计一个操作，需要同时规定正常时发生什么、请求不能完成时如何响应，以及已有数据是否保持。最后一项是**失败后状态**，不能仅靠出现错误提示来确认。

下表是 V0.1 的行为约定。其他版本可以选择幂等返回或更新接口，但要另行定义条件并验证；本讲先按已签署的同一约定实现。

操作情境	预期响应	需要再次查询的内容
在已登记项目下创建新任务	创建成功	任务属于指定项目
在不存在的项目下创建任务	拒绝创建	没有留下无所属项目的任务
使用已有任务编号创建不同任务	拒绝覆盖	原需求和记录保持不变
查询材料不完整的任务	指出缺少什么	已有记录仍然保留

SQLite 将数据保存在本地文件中，不需要为本讲另外运行一个数据库服务。Python 的 sqlite3 提供连接、写入和查询接口。在本课程 Python 3.11 的常见事务用法中，写入还需要提交；同一连接读到数据，并不能代替关闭后重新读取的检查。具体行为见 Python sqlite3 文档。

可以把本讲数据理解成几张有关联的表：项目表每行代表一个项目，任务表每行代表一项任务，运行记录表每行代表一次观察。**主键**用来唯一标识一行，例如任务编号。**外键**将子记录关联到已有父对象，例如运行记录必须属于某项任务。参考实现使用这些约束并启用外键检查；只画出关联、却没有在程序或数据库中强制执行，仍可能留下无所属对象的记录。

事务把一组数据库修改作为一个整体处理。本讲需要关心的结果是：一次错误请求不能留下半个新任务，也不能先破坏旧数据再返回错误。约束负责拒绝不合法的数据，事务处理负责让相关写入整体提交或回退；二者仍须用失败后查询验证。

下面用伪代码说明处理顺序：

代码块

```
读取并检查需求文件，得到要保存的内容
开始数据库事务
    尝试插入任务编号、已有项目编号、需求副本
    如果编号冲突或项目不存在：回退此次写入，报告失败
    否则：提交写入，再报告创建成功
```

边界检查：如果程序先删除 A，再尝试创建新 A，因新需求文件无效而报错，会有什么损失？原任务可能已经丢失。因此应尽早检查输入，并保证相关数据库修改具有整体失败的语义。不能只以“报错符合预期”判定通过。

四、用红绿对照判断能否交付

4.1 测试必须经过真实使用条件

测试是一项可重复的实验：建立条件，执行操作，将实际结果与事先确定的预期比较。本讲的持久保存实验应跨两次程序运行：

1. 初始化隔离测试目录，登记“个人工作台”项目。
2. 通过创建命令保存任务 A 和它的要求，等待命令结束。
3. 再启动一次查询命令，按 A 读取任务。
4. 核对项目归属和要求与创建时一致。

这组操作可以先手动理解，再写成自动测试。若测试只在内存里构造一份数据，然后检查同一份数据，它没有经过真实保存与重新读取，发现不了目标缺陷。

重复编号实验继续使用任务 A：再次创建不同内容，观察命令是否拒绝，再重新查询 A。**出现错误提示与原数据未被破坏是两项需要分别检查的结果。**

实际观察	可以得出的判断
创建成功，重启后找不到 A	持久保存要求未满足
重复创建被拒绝，但原要求已改变	拒绝响应正确，数据保护仍然错误
重启后原要求仍在，重复请求未改变记录	这些已执行的验收项满足约定

测试中的**断言**就是对实际值作明确判断，例如“读回的需求等于创建时需求”。只检查“有内容”弱于检查“内容准确”。测试还要使用独立数据目录，避免上次留下的 A 让这次未保存成功的程序看起来正确。跨进程调用可用 Python 的 subprocess；如何组织断言可查 unittest。具体代码由实践逐步形成。

4.2 先证明能发现问题 再核对修复



验证流程：复验失败保留记录并回到修复

先根据 Spec 创建测试，在目标能力尚未实现时运行并保存失败；再请 Codex 实现，用同一输入和预期复验。这个过程称为**先红后绿**：红表示检查失败，绿表示通过。

例如，第一次创建后重启无法查到 A，测试失败；增加持久保存后，同一测试查到原要求，转为通过。这样的前后对照说明，这项检查能够识别并确认目标行为的变化。

起始失败应来自目标能力缺失。Python 无法启动、测试语法错误或实际运行了零个测试，说明实验条件未成立，应先排错并保留记录，不能当成有效的业务反例。

“同一测试”还要求检查标准保持一致。即使命令文字没变，删掉关键断言、跳过失败用例或换到另一数据目录，都可能使结果变绿却没有修好问题。复验时要核对测试内容的 Diff、代码位置和运行条件。如果确实发现测试写错，应保留旧记录、解释修订原因，再用修订后的合理标准建立有效的前后对照。

先红后绿说明检查能够发现目标缺陷，并在修复后通过。接受本次交付还要核对需求覆盖、实际 Diff、失败后数据与人工复核。

运行记录应保存命令、输出、时间和退出码。**退出码**是程序结束时返回的数字；0 通常表示该命令按约定成功，非零需要结合输出判断原因。保存一份失败输出的操作可以成功，因为它准确记录了另一条命令的失败。

Diff 是修改前后内容的对照。对任务保存功能，核心变化可以这样概括：

```
代码块
1  - 任务只保存在当前进程的内存中
2  + 任务写入本地存储，重新启动后按编号读取
```

这是设计变化的文字示意，不是可运行代码。阅读实际代码差异时，应找到它对应的写入、读取与错误处理，并检查是否仍在约定文件范围内。

4.3 分别判断命令结果与人工验收

假设验收测试退出码为 1，说明该次检查失败。随后把它的真实输出存进工作台，追加命令退出码为 0，说明记录保存成功。这两个数字回答两个问题，应同时保留。

观察对象	示例结果	可以说明什么
被记录的测试命令	返回码 1，查询不到任务 A	该次测试失败，继续查失败原因
追加记录命令	返回码 0，生成记录编号	工作台接收了这条记录
再次查询记录	仍能读到返回码 1 和失败输出	保存过程没有把原失败改成通过
人工验收	退回修复	该功能尚未满足这项要求

同理，状态查询成功也不意味着任务验收通过。参考实现用 ok 表示本次查询及所要求的检查是否满足，用 evidence_complete 表示记录是否形成规定的完整性关系，用 acceptance: pending_human_review 表示还要由人审核。使用带任务编号和完整性要求的查询，才能检查指定任务；无参数的总览不能代替它。

验收摘要应回答四件事：按哪份要求检查、实际改了什么、实际检查了什么、还有什么没证明。例如：“重启查询通过；重复编号后的原记录保护尚未复验，因此暂不接受全部交付。”这比“全都正常”更能帮助接手者继续处理。

运行检查以后，按五个操作试用自己的程序，并让同伴依据 Spec 复核。例如，自己验证了单次保存，同伴补查重复编号；双方可以明确哪些要求已验证，哪些仍缺依据。

验收可以是接受、退回或依据不足。程序可用但记录缺失时，应说明材料不完整；材料齐全但重复创建仍覆盖数据时，应说明功能缺陷。把两者都写成“通过”会让接手者失去判断依据。

五、用新工作台完成第一次自举

5.1 先采集原始记录 再导入新工作台



自举流程：比较导入前后状态并保留原始失败

林舟先遇到一个看似绕不开的问题：工作台还没做出来，怎样用它记录开发工作台的过程？答案在时间顺序里。建设之前，人先确认需求、运行测试，并用已有的 evidence.py 采集工具保存命令、输出、时间与退出码；记录能力做出来以后，再把这些原始材料导入。采集工具是现成支架，你要建设的是项目、任务和证据之间的关联与查询。



先看能力怎样建成，再看原始记录怎样归属到自身建设任务。两条线在“用新工作台登记自身”处接合。图示是实践路径，不是运行成绩。

程序可用后，再登记“个人工作台”项目，建立“开发 V0.1”任务，并通过配套导入工具调用自己实现的记录追加命令，导入先前保存的材料。用新工具记录自身建设的过程，本课称为**自举**。

回看开发前保存在 lesson-01-submission/02-design.md 的个人设计：五个操作是否解决了原来的两个痛点？从自己的真实失败或复核中，挑出一项下次可以复用的做法，写清适用条件、步骤、检查办法，以及原始记录的位置。例如，“保存成功以后，应从新进程读回内容”。把它先作为候选方法写入 lesson-01-submission/07-revision.md，按手册中的学习记录要求保留依据，再用另一项相关检查验证它是否有用。

5.2 用自己的证据完成复核

实际使用时，逐项检查：重启后读回、重复编号被拒绝且旧数据不变、向不存在任务追加记录被拒绝且不留新记录、缺少证据时如实报告并保留已有材料。按手册中的学习记录要求保存实际结果，不要把讲义中的预期填成运行结果。

开发前的第一次签字确认“我选择做什么，接受什么范围”；复核后的**第二次签字**确认“依据这些结果，我接受哪些部分，还有哪些问题”，并写明接受、退回或依据不足。两次签字对应不同责任，Codex 可以整理文字，个人决定必须由你作出。请同伴依据 Spec 补查一项你可能漏掉的行为，保留反馈、修订前后内容和回应。尚未取得反馈就写“待复核”。

手册将实践分成八步：准备隔离区、签署需求、建立有效红灯、授权实现、审查并复验、自举、故障检查、复核提交。每一步都有要保存的记录与进入下一步的条件。线上精讲负责理解问题和关键示范，完整实践可以分次完成。

六、从这次交付走向可复用工作台

完成第一次自举后，再看工作台为什么还要继续建设。Harness 组织执行，记忆系统让历史参与下一事项，流程蒸馏把真实失败中的有效做法整理为可复用方法。本讲只建成记录基础；以下用同一个“重启后丢失”案例说明后续方向。



目标工作方式：经验经复验和具名确认后参与下一事项

6.1 执行怎样从手工组织走向受控运行

Harness 可以理解为围绕 AI 执行任务的一套运行与控制机制。 它把任务输入、工具、执行状态、检查和停止条件接在一起，使一次工作可以按约定推进。这里的“控制”既包括提供可用的执行环境，也包括落实边界、处理失败和组织恢复。

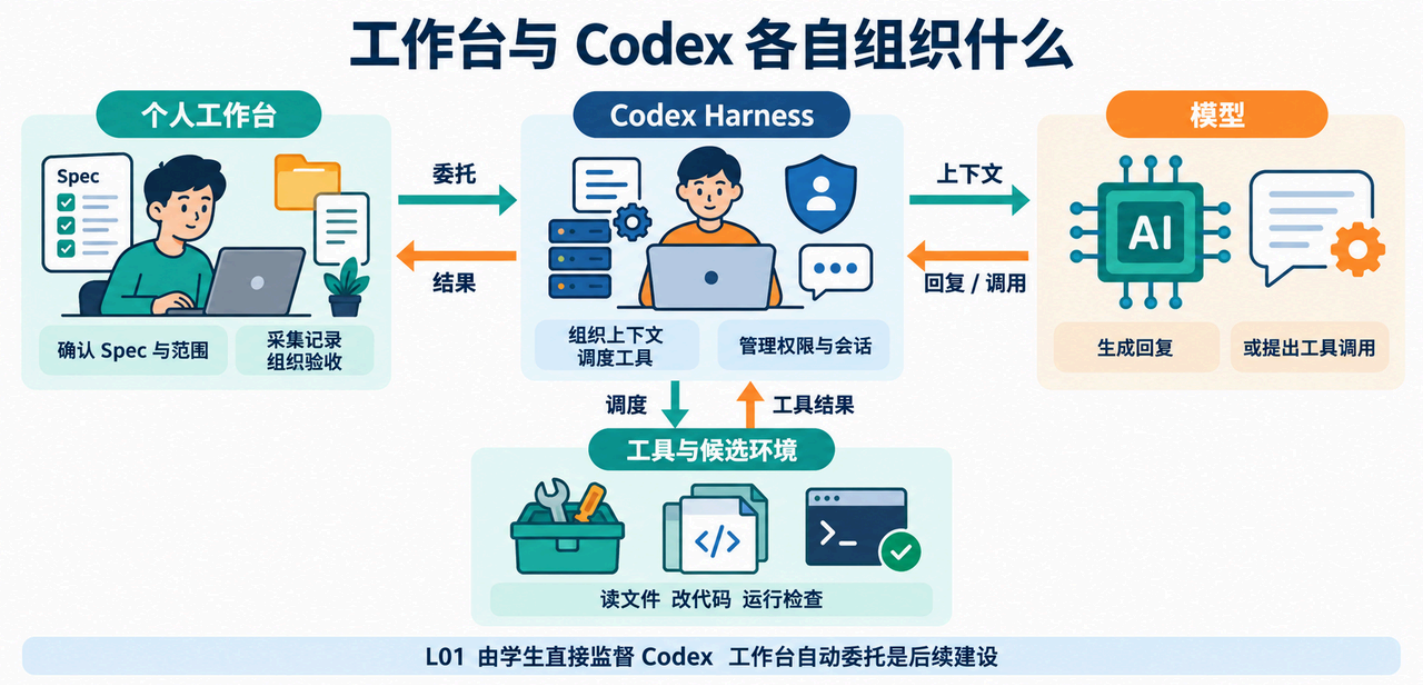
以“让任务关闭后仍能查回”为例，一个完整的工作台 Harness 要做这样的工作：执行前提供确认后的需求和项目位置；限定本次可改的文件；组织 Codex 修改；运行重启查询检查；失败时保留输出并进入返工；缺少授权或无法判断时停下来交人处理。

写下“请遵守范围”，只是提出要求；执行机制还要检查这个范围是否被遵守。 同样，“必须运行测试”只有在检查真的执行、结果被读取，并影响后续处理时，才成为流程中的有效约束。自动重试也需要次数或时间上限，以及退出后的状态说明，否则只是把同一个问题重复得更快。

关于长任务的工程实践，Anthropic 用进度文件、逐项功能检查和会话交接来改善连续执行，说明了执行环境与交接机制的重要性，可参阅 Effective harnesses for long-running agents。这里借鉴的是组织长任务的方法，课程中的具体边界仍由我们的需求与检查确定。

6.2 工作台与 Codex 的运行分工

当你直接委托 Codex 开发工作台时，模型提出回复或工具调用，Codex Harness 组织上下文、按权限调度工具，并把执行结果带入后续请求。



图中预告 L04 的分工：工作台组织委托与验收，Codex 组织模型和工具往返。L01 新建程序只记录与查询，由你直接监督 Codex；图中的自动调用和客户交付尚未接入。

6.3 保存的记录怎样参与下一次任务

假如每次启动任务都要重新解释项目、重新寻找失败记录，即使单次执行很规范，长期开发仍然费力。记忆系统解决的是**哪些信息应该留下，以及下一次在什么时候取出来使用**。

先为这次开发建立任务 A，保存确认后的要求，把首次失败与修复后的运行分别挂在 A 下。此后你可以从 A 找到原要求、历次尝试和未完成的检查。项目说明“在维护什么”，任务说明“这次完成什么”，运行记录说明“实际尝试过什么”。三者各有归属，项目中的一个成功结果才不会被误当成所有任务都已完成。

存储只是记忆系统的基础。 有一百份日志，但下一项任务找不到相关记录，记忆就没有真正参与工作。还需要按项目和任务组织、提取简明进度、保留来源，并在执行前选出相关内容。把所有旧聊天都塞进当前输入，既可能干扰注意，也可能把已经过时的决定带回来。上下文是本次提供给 AI 的信息，长期保存的材料要经过选择才成为当前上下文。关于这一区别，可参阅 Effective context engineering for AI agents。

前文的需求快照解决了“当时依据什么”；下一事项还要选择适用材料，分清事实、推测和已经失效的规则。

又如，日志里写着“怀疑读错数据目录”，这是一条待验证的推测；新进程确实读不到 A，才是观察到的事实。记忆中要保留这种区别。否则一次排错猜测经过摘要，就可能变成下次执行默认相信的“项目事实”。

6.4 从真实失败提炼可复验的方法

假如工作台记住了“任务 A 曾在重启后丢失”，它保存了一次经历。下一次开发类似功能时，我们更希望它能提醒并落实：“需要长期保存的数据，应在新进程中重新读取后再验收。”这就从记录一次

事件，走向提炼一种方法。

本讲把**从实际工作中提取有效步骤、适用条件、检查与停止规则，再整理为可复用方法**的过程，称为工作流程蒸馏。这个词强调提炼：原始过程常有猜测、绕路和偶然条件，要判断哪些经验值得留下，以及它们在什么范围内成立。

1. **保留原始经历。** 创建命令显示成功，同一进程能查询，新进程却读不到。把命令、输出和当时的代码改动保留下来。
2. **解释失败原因。** 查明测试只读取内存，没有经过持久保存。由此定位到“检查绕开了真实使用条件”。如果实际原因是数据目录不一致，就应记录那个原因，不能套用预设结论。
3. **提出可复用的方法。** 对“程序退出后仍要保留”的需求，先使用独立数据目录创建，再结束进程，通过新进程读取，并核对内容。
4. **让方法接受下一次检验。** 后面增加运行记录保存功能时，用相同思路检查输出能否在重启后查回。记录它发现了什么，也记录它仍然漏了什么。
5. **确认后进入日常流程。** 将这一检查写入验收模板或测试入口，后续相关任务能够选用。条件变化时保留版本和修订理由。

这里提炼出的是“跨进程检查持久化”的方法，不是“所有软件都必须用 SQLite”。前者来自使用要求，后者只是本次方案。把一次实现中的偶然选择当成普遍规则，会让工作台积累越来越多不适用的限制。

一条候选经验需要经过判断和复验，才能成为默认规则。 Codex 可以协助归纳，你负责确认它是否符合事实、是否适用于下一类任务。应保存原案例和反例，避免把未经验证的建议直接加入质量门。这里更新的是规则、上下文和程序化流程，不涉及模型参数训练，也不代表模型会自动进化。

把三者连起来：记忆保留“重启后丢失”的真实经历；人和 Codex 提炼出跨进程验收方法；经过复验与具名确认，后续任务采用这一方法，Harness 组织执行，新的结果再回到记忆。若复用失败，就追查适用条件并修订或停用该方法。

这也体现了 FDE（Forward-Deployed Engineering）的现场、交付、能力三循环：在使用现场发现问题，完成一次有依据的交付，再把可复用能力带回下一次任务。L01 先保存自身建设过程，并手工提炼一项候选方法；完整的自动执行、记忆取用与流程复用将在后续建设中验证。

6.5 全课能力图与后续学习位置



周宁以后提出库存、订单和采购问题时，林舟会反复使用这些能力。它们不是必须依次通过的十二道工序：Prompt 说明这次要做什么，Context 提供本次需要的规则、事实和状态；Agent 在 Harness 提供的运行条件与控制下使用工具，也可以按 Skill 复用方法。Loop 管理重复修复，Graph 表达分支、移交和等待人的决定。

Memory 保存经过治理的经验，RAG 为当前任务检索材料、组织上下文并保留引用，采用哪些仍由人判断。最小领域本体说明订单、采购单、候选和审核各指什么，约束 Spec、Eval 与 Graph 中的动作。FDE 贯穿现场、交付和能力积累；AI4AI 在本课程中指借助 AI 建设和改进 AI 工作系统，不指训练模型权重。

能力	林舟在故事中用它解决什么	主要阅读位置
Prompt、Context	交代目标，提供足够且有效的材料	本讲起步；L02、L06、L10、L11 继续展开
Agent、Skill、Harness	组织行动，复用方法，约束执行并读取反馈	L02、L04、L06；L11 进入 Subagents
Loop、Graph	有依据地返工，等待、分支、回退与恢复	L09 先做一次返工；L10、L12 分别建设
Memory、RAG	找到适用经验，带来源交给下一事项	L10 处理任务内接续；**L15 系统讲授 Memory / RAG**
本体驱动	明确对象、关系、允许动作与业务约束	L12 最小本体卡；约束具体交付，非完整本体平台

FDE、AI4AI	从客户问题积累可复用方法，用 AI 改进工作系统	FDE 贯穿全课，L04 换挡、L06 展开；AI4AI 在 L01、L09、L16 前后呼应

图中是全课建设路线。先用本讲记录基础接续任务，再逐步加入执行控制、检索与流程复用；是否形成闭环，要由跨事项采用、Eval 与人审结果证明。

6.6 迁移练习与下一讲准备

6.6.1 条件改变后重新判断方案

从一人变成多人。甲乙同时读取旧任务，甲先保存修改，乙稍后保存，可能覆盖甲的结果。先列出谁在什么时间修改哪个对象，再决定是禁止原地更新、引入版本冲突检查，还是限制写入次序。即使数据库把两次写入依次完成，也可能发生后写覆盖前写的业务损失。应以“双方改动与冲突怎样处理”为验收目标，不能仅凭没有报错判断并发安全。数据位于哪台机器、是否需要远程访问、谁有修改权限也都要重新确认，人数本身不足以决定换哪种数据库。

已有保存函数是否可以复用。假设它收到重复编号就覆盖旧值，而本次 Spec 要求拒绝覆盖。函数名称虽然是“保存任务”，行为却不符合本次规则。可以复用其底层连接能力，另外实现符合要求的创建操作；也可以修改原函数，但要检查已有调用者是否依赖覆盖行为。选择依据包括行为差异、影响范围、依赖成本和可执行的正反测试，不能只看代码行数。

练习：任选一种变化，写清一个实际条件、一项选择、一种代价和一项能推翻自己方案的实验。先独立写，再与 Codex 讨论。保留首次选择与修订理由。

6.6.2 换一个项目重新定义验收

假如你还要维护一个文档校对项目。项目、任务、需求副本与历次运行可以复用，因为仍要保存“这次按什么要求处理，做过什么检查”。文档正确性的判断需要重新定义。

例如，编辑要求“统一术语，但不改变引用原文”。可以约定全文使用指定术语表，同时逐条核对引用与来源。这时仅检查全文不含旧词会产生错误：引用中可能必须保留原词。程序可辅助定位旧词和引用位置，编辑要确认哪些位置允许替换，并复核语义是否改变。

请写一个正常例和一个错误例，说明哪些检查可以程序化、哪些需要人读。再回答：工作台保存了一份校对报告，是否足以证明引用没有改错？所需证据应回到文档版本、实际差异和引用核对结果。

6.6.3 带一条真实规则进入下一讲

本讲保存任务与运行记录。L02 把长期项目规则写进 AGENTS.md，再用实际请求检查协作行为；L03 补结构化需求与解析；L04 形成工作台 V0 并首次交付库存导出。L05～L16 持续通过 FlowERP 的需求与失败升级工作台，L16 用此前未实现的受控小需求检验迁移。

课前准备一条真实规则，例如“创建任务不能覆盖旧编号”，说明触发条件、期望处理和怎样检查。下一讲继续用请求与观察判断规则是否有效。

附录 A：课程成品与产品边界

以下保留参考成品，帮助理解课程终点。日常开发在个人工作台中组织，企业员工在 FlowERP 中处理业务；图中完整界面不是 L01 的交付要求。

A.1 个人研发工作台给开发者使用

打开工作台，你首先选择正在开发的代码项目，再提出这次要完成的任务。任务里有确认过的要求、允许修改的范围，以及用来检查结果的办法。完整工作台据此组织 Codex 执行，汇总实际改动与检查结果，供你决定接受还是返工。

工作台组织的是一项开发任务从提出到完成的过程。 它的日常使用者是负责开发与交付的人。你可以同时维护几个项目，但每项任务的要求和结果要有明确归属，失败以后也要能在原任务中继续处理。



工作台参考成品

项目入口提供开发上下文，任务列表记录正在处理的工作，验收入口集中需要作出决定的结果

真正接手一项工作时，你会进入任务详情。比如，某项任务的创建功能已通过检查，重启查询却仍然失败。任务详情应保留这两类结果，使你知道下一步要修复保存或读取过程，而不是从头解释整个项目。

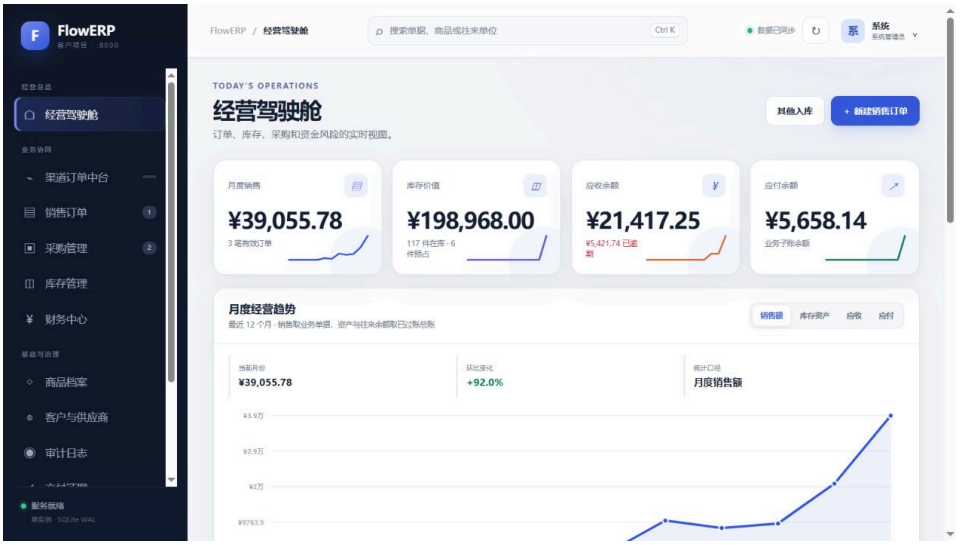


参考任务的复验与审核页面

截图中的操作是复验已有任务，当前等待人审；它展示记录的组织方式，不代表这一轮新开发了功能

A.2 FlowERP 给客户处理业务

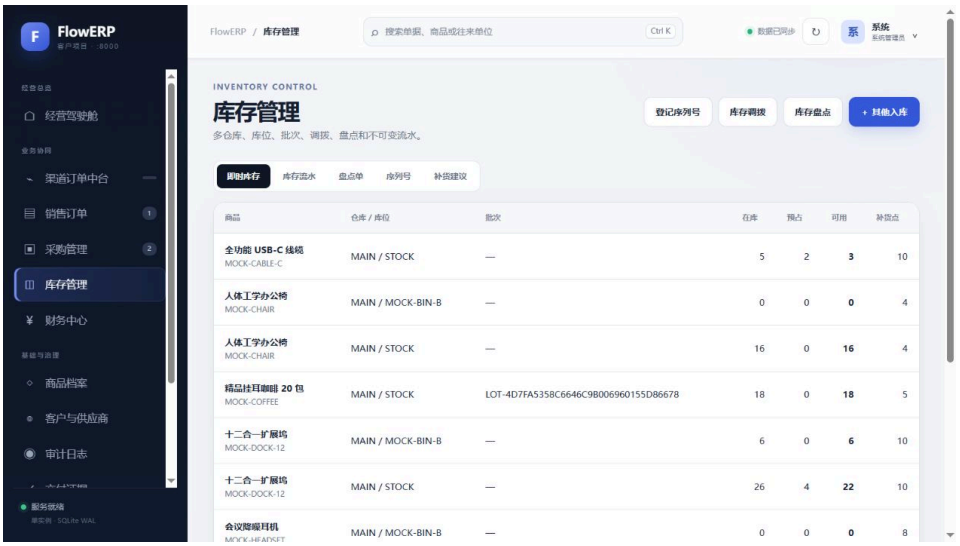
FlowERP 是电商企业使用的业务系统，覆盖商品、库存、订单和采购。ERP 是“企业资源计划”的缩写；在本课程中，重点是这些业务怎样共享数据、彼此影响。



FlowERP 参考成品，使用课程演示账套。

企业员工在这里处理业务，开发者在个人工作台中组织这些功能的建设

例如，销售人员在 FlowERP 中创建订单，系统为订单预留库存。以后客户要求“取消订单后释放库存”，你在工作台中建立开发任务，请 Codex 协助修改，再用 FlowERP 的实际行为检查结果。



FlowERP 参考库存页面

它展示客户产品将要承接的业务信息；本讲不实现库存页或库存导出

因此，两项成果有明确关系：**工作台是首要建设成果，FlowERP 是持续开发而形成的伴生客户产品，也是工作台的验证场。** 产品需求使工具设计有具体用途；产品失败暴露工具能力的不足；改进后的工作台再接受下一次交付检验。

截图来源：2026-09-05 本地参考程序。图 9 月度销售 ¥39,055.78 表示当月 3 笔未取消订单的金额，包含不同订单状态，不能解释为已收款收入。演示数据与来源见截图说明。

附录 B 本讲课程合同

以下保留正式大纲的四项要求，供完成实践后逐项核对。本讲对应课程目标 CLO-1：识别结构、约束、验证入口和责任边界。

- **核心内容**：开课时没有个人工作台、WORKBENCH_SPEC.md 或验收测试；你从一句模糊诉求和真实痛点出发，利用 Codex 完成需求访谈，由你决定范围并签署首份 Spec，再把验收项转成红灯测试，开发初始化、项目登记、任务账、命令证据账和状态检查。
- **演示结果**：“帮我搭建个人 AI 研发工作台” 经历“原始诉求 → Codex 追问 → 你取舍 → WORKBENCH_SPEC.md → 验收测试红灯 → Codex 实现 → 同一测试绿灯”，最后由新工作台记录自己的建设证据。
- **课内增量**：你亲手形成并签署首份 WORKBENCH_SPEC.md、从验收项生成失败测试、开发出可运行的 Workbench V0.1，并完成第一次自举记录；本讲不接入、不开发 FlowERP。
- **通过标准**：Spec 每项关键要求能回指原始痛点、Codex 建议和你决定；测试能回指验收项；必须保留实现前红灯、范围内 Diff 和实现后绿灯；你能用自己开发的命令完成自举，并说明 L04 才会通过“个人工作台 + Codex”开发 FlowERP。

附录 C 术语查阅

术语	本讲中的意思
Harness	围绕 AI 任务的运行与控制机制，组织工具、检查、返工和停止
记忆系统	保存、组织、选择和更新跨任务信息，让相关经历参与当前工作
工作流程蒸馏	从真实经历提炼有适用条件、能复验的规则和方法，供后续使用
Spec	开发前确认的需求说明，写清做什么、边界和检查办法
Diff	修改前后的内容对照，用来查看实际改动
ID	稳定编号，让项目、任务和运行记录能对应起来
快照	当时内容的副本，用来查清当时依据的要求
红灯／绿灯	检查失败／通过，具体原因要查看实际输出
自举	用新工作台记录它自己的建设过程
持久化	程序结束后数据仍保留，下次启动可以读取
写集	本次允许修改的具体文件范围
命令证据	实际运行的命令及输出、时间、退出码等记录

附录 D 来源与配套阅读

原稿来源核验日期为 2026-09-06，参考截图为 2026-09-05。以下保留原文出处，供进一步查阅。正式要求以课程大纲 L01 为准；实践命令与提交路径见同目录实践操作手册。

· Anthropic：Effective harnesses for long-running agents：执行环境、进度记录与跨会话交接。

· Anthropic：Effective context engineering for AI agents：当前上下文与长期笔记的组织。

“Harness + 记忆系统 + 蒸馏后的工作流程”是本讲用于解释工作台的设计框架；“流程蒸馏”指工作方法的提炼与验证，不作为上述文章提出的统一定义。

· Python 3.11：sqlite3：本地持久化、提交与事务处理。

· Python 3.11：unittest：测试用例与命令行运行。